

Wavetable synthetizer

Abstract

We created polyphony wavetable synthetizer that allows you to select knob and put a LFO and envelope modifier on it. We have also built a nice visualization of the actual wavetable, LFO and envelope. Furthermore, we included several wavetable presets and even loading from wav files. The wavetable oscillator also, other than the usual morphing between waveforms has a unison voices with detune.

1. Introduction

Warning:

Write an introductory section, giving an overview of the project and explaining the motivation behind it.

The report should be a maximum of 6 pages long.



Figure 1: Our final result of the wavetable synthetizer.

1.1. Motivation

In the early 2010s I was very impressed by the popular wavetable synthetizer plugin Massive from Native Instruments ([Figure 2](#)). At that time it provided new functionality to grab and drop LFO or envelope onto any knob and select how it affects given knob. This meant that the user had even more freedom to create new sounds and had been given more power control over the sound. Which lead to creative sounds and was often used for the wobbly sounds in dubstep.

Since the audio plugins are often very costly, especially for a teenager, I always wonder how hard it would be to create plugin like that.



Figure 2: Native Instruments Massive. Plugin that was the initial inspiration and motivation for this project. Especially the feature of putting LFO and envelope onto any knob.

2. Background

Warning:
Write here about the theoretical background of the work, the algorithms used.

Nearly all of the components are own made, with the exception of the filter, which is the one from lectures.

2.1. Wavetable

As is common, wavetable synthesizers usually contains single period of the used waveforms. However, since I wanted to try loading from files and then replaying the song, it supports waveforms with “different period counts” (i.e., sample block from the file that fits into the waveform buffer).

The other significant wavetable feature “position” or “morph” between the individual waveform is done by using linear interpolation. We know about the pre-calculation of the morphing, but this wasn’t our priority and lerping was sufficiently fast.

The wavetable volume controls just the volume of the waveform, while you output volume is volume after applying unison and filter.

The unison is done in stereo with minus detune on left channel and positive on right channel for each voice number.

The actual processing is sample-based, rather than block-based.

The wavetable contains visual representation of the actual waveforms in the wavetable based on selected preset or from selected file. The visualization highlights the currently selected waveform and the morphing between them. The visualization draws each waveform in different color to easier recognize each waveform.

2.2. “Effects”

This is what we call the LFO and envelope, that can affect other knobs.

The inspiration for this arose from Massive (Figure 2) and the GUI then was inspired by Pigments (Figure 3) to have select affected component and have a row of modifier effects.



Figure 3: Arturia Pigments. Inspiration for the row of effects.

You can press the button “select component” and then click (select) which knob you want to affect. Unfortunately not all knobs can be affected by this (For example envelope knobs don’t have that and unison voice and detune). Selecting a component also loads it’s current multipliers of the effects for the selected component.

2.3. LFO

The GUI visualization of LFO is also inspired by the Pigments (Figure 3). The LFO contains the usual frequency knob with a knob for offset, type selection and its visualization of the current setting.

2.4. Envelope

We used the usual ADSR (attack, decay, sustain, release) envelope with its visualization. The envelope can be either edited with the knobs or using the visualization, which also allows to make the envelope non-linear.

3. Implementation

Warning:

Please describe your implementation. You can illustrate the implementation with pseudo-code and/or diagrams. The following example of pseudo-code illustrates the programming of a delay line.

3.1. Parameter

I think it is fair to start talking with the Parameter class, as this is meant to be abstraction for the “parameters” or “knobs”, which smooths all the value settings and provides way to set effects for each parameter. The smoothing is done by the juce SmoothedValue class. The originalValue is the value set from the GUI and smoothedValue is the result smoothed value after the effects processing.

The effects are done the following way: each parameter stores it effects that affects the parameter together with its the effect id, multiplier, reference and lambda to calculate the parameter value from the effect (EffectEvaluator) to generalize for the calculation of the value.

```
using EffectEvaluator = std::function<FloatType(FloatType previousValue, FloatType originalValue, FloatType dspMultiplier, DSP<FloatType>& dsp)>;
```

The above mentioned values are store in the following vector:

```
std::vector<std::tuple<juce::String, FloatType, std::reference_wrapper<DSP<FloatType>>, EffectEvaluator>> effects;
```

This also means that it is not store in the plugin preset. The main issue would probably be the reference to the dsp.

The value when need gets calculated by iterating the effects vector, applying the lambdas of each effect on the originalValue and getting the final result.

The DSP class is meant to be an abstraction for a dsp can that affect the parameters value and each dsp would move it's state independently on the parameter. It is a pure virtual function that is implemented by the effects – LFO and envelope.

This technique results in a bit chaotic and complicated approach of setting the value in AudioProcessor.cpp especially the functions: getParameterEffect() – to get the lambda for calculating value from the dsp, updateParameterCoefficients() – assign current modifier values to the gui from the parameter and applyParameterEffect() – to apply the effect on the correct parameter (to all voices, to lfo, ...).

3.2. Wavetable

The class is basically simple wrap-around, with some helper methods for loading from preset of file.

```
static constexpr std::size_t SampleSize { 2048ul };
std::vector<std::array<float, SampleSize>> wavetables;
```

3.3. Synth Voice

Files: WavetableSynth{.h, .cpp}.

The voice is processed sample by sample rather than per block basis.

The increment from the wavetable is $\frac{\text{noteFrequency}}{\text{originalSampleFrequency}} * \frac{\text{sampleSize}}{\text{sampleRate}}$, where sampleSize is the size of the array that holds one waveform “period” and originalSampleFrequency is a value, that enables playback of a longer sample (song) at default note (A4).

The unison voices are accomplished simply by holding phases and increments for the individual voices on each channel.

```
static constexpr int Channels = 2;
std::array<std::vector<float>, Channels> unisonPhases;
std::array<std::vector<float>, Channels> unisonIncrements;
```

For these the most important functions are `updateUnisonIncrements()` and `renderNextBlock()`. Each voice increment is number of voice * 0.1 * detune with left channel being negative and right one being positive values.

The output value from unison is they divided be the count of voices to avoid very loud result on low detune. We also added random initial phase for the unison voices to avoid oscillation effect in the `startNote()`.

Afterwards, the filter for each channel is processed and combined together using the panning value.

3.4. LFO

The LFO provides the usual types: sine, square, triangle and sawtooth wave. The LFO uses the aliased version and calculates the value on the go (no wavetable used for the lfo). The frequency of the LFO is a public Parameter to allow changing it using the effects with selected component.

```
case Waveform::Sine:
    value = std::sin (2.0f * juice::MathConstants<float>::pi * phase);
    break;

case Waveform::Square:
    value = (phase < 0.5f ? 1.0f : -1.0f);
    break;

case Waveform::Triangle:
    value = 4.0f * std::abs (phase - 0.5f) - 1.0f;
    break;

case Waveform::Sawtooth:
    value = 2.0f * (phase - 0.5f);
    break;
```

3.5. Envelope

In the current implementation only the “Envelope A” affects the volume of the synth voice, while the “Envelope B” is only used for the custom effects.

4. Discussion

Reflect on the project as a whole. Reflect on what was difficult and what did you learn? In what direction could the work be taken in the future?

As synthetizer are quite robust plugins there is a lot still that can be improved or added. Many wavetable synthetizers provide their own reverbs, more filter, but we could also add multiple oscillators instead of just one.

- Pre-calculation of waveform lerps